

鹰眼长空—开发文档

Contents

1	鹰眼长空—开发文档	2
1.1	1. 项目概述	2
1.2	2. 环境搭建	2
1.2.1	2.1 依赖	2
1.2.2	2.2 配置 config.json	2
1.2.3	2.3 启动	3
1.2.4	2.4 测试数据	3
1.3	3. 完整目录结构	3
1.4	4. 分层架构	5
1.5	5. 各模块详解	5
1.5.1	5.1 core/state.py —全局状态	5
1.5.2	5.2 core/config/config_manager.py —配置管理	5
1.5.3	5.3 core/io/data_loader.py —数据 I/O	5
1.5.4	5.4 core/prediction/prediction.py —预测算法	6
1.5.5	5.5 core/ai/ai_service.py —AI 服务	6
1.5.6	5.6 services/data_service.py —数据服务	6
1.5.7	5.7 services/predict_service.py —预测服务	6
1.5.8	5.8 services/backup_service.py —备份服务	6
1.5.9	5.9 views/ —HTTP 视图层	7
1.5.10	5.10 frontend/ —前端详解	7
1.6	6. 关键数据流	7
1.6.1	6.1 启动流	7
1.6.2	6.2 预测流	8
1.6.3	6.3 备份恢复流	8
1.7	7. API 完整参考 (13 个端点)	8
1.7.1	数据管理	8
1.7.2	预测 & AI	9
1.7.3	备份管理	9
1.7.4	其他	9
1.8	8. 添加新功能指南	9
1.8.1	8.1 新 API 端点	9
1.8.2	8.2 新页面	10
1.8.3	8.3 新算法	10
1.8.4	8.4 新检测平台	10
1.9	9. 前端开发参考	10
1.10	10. 调试	10

1 鹰眼长空—开发文档

面向新开发者的完整上手指南，读完即可参与开发。

1.1 1. 项目概述

鹰眼长空是一个**双模块**无人机轨迹监测系统：

模块	端口	职责
trajectory_reconstruction	5000	轨迹加载、3D 可视化、预测、AI 策略、运动学分析
trajectory_recognition	5001	轨迹模式识别与分类（框架阶段）

两个模块通过根目录 data/ 和 reports/ 共享数据，导航栏可一键跳转。

技术栈： Python Flask · pywebview · Three.js (本地) · ECharts (本地) · OpenAI 兼容 AI 接口

1.2 2. 环境搭建

1.2.1 2.1 依赖

pip install flask pywebview requests

1.2.2 2.2 配置 config.json

```
{
  "ai": {
    "provider": "siliconflow",
    "api_key": " 你的密钥",
    "url": "https://api.siliconflow.cn/v1/chat/completions",
    "model": "Qwen/Qwen2.5-7B-Instruct"
  },
  "detection_methods": {
    "visible": {"name": " 可见光", "color": "#FF3B30", "visible": true},
    "infrared": {"name": " 红外", "color": "#34C759", "visible": true},
    "radar": {"name": " 雷达", "color": "#FFCC00", "visible": true}
  },
  "prediction_settings": {
    "min_points": 1, "max_points": 20,
    "default_points": 6, "time_step": 0.5
  }
}
```

ai 字段兼容任意 OpenAI Chat Completions 接口，切换模型只需改 url 和 model。支持示例：

```
{
  "ai": {
    "url": "https://api.deepseek.com/v1/chat/completions",
    "model": "deepseek-chat"
  }
}, {
  "ai": {
    "url": "https://api.openai.com/v1/chat/completions",
    "model": "gpt-4o-mini"
  }
}, {
  "ai": {
    "url": "http://localhost:11434/v1/chat/completions",
    "model": "llama3"
  }
}
```

1.2.3 2.3 启动

```
python main.py          # 默认同时启动两个模块（桌面窗口）
python main.py --headless # 纯 HTTP（两个模块都启动）
python main.py recon     # 仅重建分析 → :5000
python main.py recog     # 仅轨迹识别 → :5001
python -m trajectory_reconstruction --headless
python -m trajectory_recognition --headless
```

1.2.4 2.4 测试数据

data/fact/ 下有 3 组虚构轨迹（各 60 点）。被误删时：

```
git restore data/fact/
```

1.3 3. 完整目录结构

```
drone/
  main.py          # 统一入口
  config.json      # 运行时配置（不入库）
  requirements.txt

trajectory_reconstruction/
  app.py           # ===== 重建分析模块 =====
  __main__.py      # Flask 应用工厂（create_app）
                   # python -m 独立入口

  core/            # ---- 领域逻辑层（纯函数） ----
    state.py       # 全局内存缓存 detection_methods
    config/
      config_manager.py # 配置读写、校验、默认值
    io/
      data_loader.py  # .dat 文件 I/O
    prediction/
      prediction.py    # 线性外推预测算法
    ai/
      ai_service.py    # 大模型 API 调用

  services/        # ---- 业务逻辑层 ----
    data_service.py  # 数据加载/刷新/清理
    predict_service.py # 预测编排 + 参数校验
    backup_service.py # 快照备份/列表/恢复/删除

  views/           # ---- HTTP 视图层（薄层） ----
    __init__.py     # 蓝图注册 + 错误处理
```

```

main.py                # 5 个页面路由
api.py                 # 数据 & 备份 API
api_predict.py         # 预测 & AI API
api_report.py          # 报告保存 API
analysis.py            # 运动学分析数据 API

frontend/              # ---- 前端资源 ----
  pages/               # Jinja2 模板
    base.html          # 基础布局 (导航栏+toast)
    index.html         # 总览 (全屏 3D)
    predict.html       # 预测
    analysis.html      # 分析 (ECharts)
    ai.html            # AI 策略
    data.html          # 数据管理
  static/
    css/main.css       # 全局样式 (Apple HIG)
    js/
      lib/             # 第三方库 (本地, 离线可用)
        three.module.js # Three.js
        OrbitControls.js
        CSS2DRenderer.js
        echarts.min.js  # ECharts
      common/
        toast.js        # Toast 通知组件
        utils.js        # 共享工具函数 (lerp)
      pages/           # 每页独立 JS 模块
        dashboard.js    # 3D 场景/轨迹/动画
        predict.js      # 预测 + 3D 预览
        ai.js           # AI 策略交互
        data.js         # 备份管理

trajectory_recognition/ # ===== 识别模块 =====
  app.py               # Flask 应用工厂 (端口 5001)
  __main__.py         # 独立入口
  frontend/
    pages/index.html  # 主页面
    static/js/pages/
      recognition.js  # 状态面板 + 数据展示
  features/__init__.py # 特征提取 (待实现)
  models/__init__.py  # 模型定义 (待实现)
  classifier/__init__.py # 分类器 (待实现)

data/                  # 共享运行时数据
  fact/                # 实际轨迹 (.dat)
  predict/             # 预测结果
  backup/              # 快照备份
reports/               # AI 策略报告
templates/             # 配置模板 (config_template.json)

```

1.4 4. 分层架构

frontend/	(HTML/CSS/JS)	← 浏览器渲染
views/	(Flask 蓝图)	← 仅 HTTP 解析和 JSON 响应
services/	(业务编排)	← 参数校验、流程控制
core/	(领域逻辑)	← 纯函数, 不依赖 Flask

依赖方向: views → services → core (单向, 不可逆)

原则: - core 不导入 Flask、services、views - services 不导入 Flask、views - views 只做参数提取和 JSON 序列化

1.5 5. 各模块详解

1.5.1 5.1 core/state.py —全局状态

```
detection_methods = {
    "visible": {"name": " 可见光", "color": "#FF3B30", "visible": True,
               "points": [[x,y,z],...], "timestamps": [t,...]},
    "infrared": {...}, "radar": {...},
}
```

"self" 键在用户上传时动态创建

AI_API_KEY, AI_URL, AI_MODEL ← 从 config.json 的 ai 字段读取

生命周期: 1. init_from_config() —读取 config.json 元信息, 清空 points/timestamps 2. initialize_data() —从 data/fact/*.dat 加载轨迹点 3. API 调用可增删改内存数据 4. save_metadata() —元信息变更后写回 config.json

1.5.2 5.2 core/config/config_manager.py —配置管理

```
ensure_config() # 确保 config.json 存在, 不存在则从模板创建
save_config(cfg) # 保存配置
```

配置文件缺失时自动从 templates/config_template.json 复制, 无模板则用硬编码默认值。_validate_config() 自动补全缺失字段。

1.5.3 5.3 core/io/data_loader.py —数据 I/O

```
load_dat_file(path) → (points, timestamps) # 返回两个列表
save_predict_data(id, pts, ts) # 保存到 data/predict/
load_default_data() → {methodId: {points, timestamps}}
```

.dat 格式: 每行 x y z t, 空格分隔。文件不存在或异常返回空列表。

1.5.4 5.4 core/prediction/prediction.py —预测算法

```
generate_prediction(points, timestamps, num_points=5, time_step=0.5)
    → (pred_points, pred_times)
```

取最后两点计算速度向量，沿该方向以时间步长生成预测点。至少需要 2 个历史点。

1.5.5 5.5 core/ai/ai_service.py —AI 服务

```
get_ai_suggestion(methods_data, api_key, url, model) → str
```

构建含轨迹数据的 prompt，调用 OpenAI 兼容 Chat Completions API，返回捕捉策略文本。超时 30 秒。

1.5.6 5.6 services/data_service.py —数据服务

函数	说明
initialize_data()	启动时加载 data/fact/ → detection_methods, 移除旧 self
refresh_fact_data()	重新加载 fact 文件 (保留 self 平台)
load_self_data(pts, ts)	创建/更新自选平台, 保存元数据
clear_all_data()	先创建快照备份, 再删除文件、清空内存
save_metadata()	持久化 detection_methods 元信息到 config.json

1.5.7 5.7 services/predict_service.py —预测服务

函数	说明
get_predict_config()	读取 prediction_settings 配置
clamp_params(n, ts)	参数约束到 min/max 范围
predict_single(id, n, ts)	单平台预测, 自动保存到 data/predict/
predict_all(n, ts)	所有可见平台 (visible=true 且 ≥2 点)

1.5.8 5.8 services/backup_service.py —备份服务

快照目录结构:

```
data/backup/20260629_143052_auto/
  manifest.json      # {created, label, methods, files}
  fact/              # 复制自 data/fact/
  predict/           # 复制自 data/predict/
  memory/            # <method_id>.dat
```

函数	说明
create_backup(label="manual")	创建快照 → 返回名称
list_backups()	列出快照 (按时间倒序)
restore_backup(name)	先 _clear_all() 再恢复
restore_all_latest()	恢复最新快照
delete_backup(name)	rmtree 删除

函数	说明
<code>migrate_legacy()</code>	启动时清理旧扁平.dat 文件

1.5.9 5.9 views/ —HTTP 视图层

文件	职责
<code>__init__.py</code>	注册 5 个蓝图 + 400/404/500 错误处理
<code>main.py</code>	页面路由: /, /predict, /ai, /data, 通过 <code>_page_context()</code> 注入数据
<code>api.py</code>	数据管理 (refresh/load/clear) + 备份管理 (list/restore/create/delete)
<code>api_predict.py</code>	预测 (predict/predict_all) + AI 建议 (ai_suggestion)
<code>api_report.py</code>	报告保存到 reports/
<code>analysis.py</code>	分析页面 + /analysis/data (速度/加速度/曲率/高度)

1.5.10 5.10 frontend/ —前端详解

CSS (main.css): Apple HIG 深色主题, 所有颜色/圆角/阴影通过 `:root` 变量定义, 按钮有 hover 上浮 + active 按压 + focus-visible 焦点环。

JS 架构: 每页独立 ES6 模块, 共享 `toast.js` 和 `utils.js`。

dashboard.js 关键流程: 1. `init()` → 异步加载 `Three.js/addons` → `buildScene()` 创建相机/渲染器/控制器/CSS2DRenderer 2. `buildLights()` + `buildGround()` + `buildGrid()` + `buildAxes()` (自定义刻度坐标轴) 3. `refreshAll()` → 遍历 `detection_methods` → `buildTrailMesh()` (CatmullRom 曲线 + 发光层) 4. 预测: `addPredLine()` 从最后一个历史点连到预测点 (消除断连) 5. `startLoop()` → `requestAnimationFrame` 渲染循环 6. 时间轴动画: `startAnim()` → 按时间戳 lerp 移动球体

predict.js 关键流程: 1. `initViewer()` → 容器内 3D 场景 (相机/灯光/网格/控制器) 2. `drawTrails()` → CatmullRom 曲线 + 球体标记 3. `runPrediction()` → POST API → 渲染预测虚线 (从历史最后点连接) + 半透明球体 4. `startAnim()` → 完整历史 + 预测轨迹动画播放

data.js 关键流程: 1. 上传: POST `/api/load_data` (FormData) → 刷新页面 2. 备份列表: `loadBackupList()` → 渲染复选框 + 信息 + 底部操作按钮 3. 批量删除: `deleteSelected()` → 遍历 `selectedBackups Set` → POST `/api/backup/delete` 4. 恢复: 仅单选可用 → POST `/api/restore_backup` 5. 手动备份: POST `/api/backup/create`

1.6 6. 关键数据流

1.6.1 6.1 启动流

```
main.py → app.create_app()
    → os.chdir(项目根目录)
    → ensure_config()           # 确保 config.json 存在
    → migrate_legacy()          # 清理旧备份文件
```

- `init_from_config()` # 填充 `detection_methods` 元信息 (`name/color/visible`)
- `initialize_data()` # 从 `data/fact/*.dat` 加载轨迹点
- `register_blueprints()` # 注册所有蓝图路由

1.6.2 6.2 预测流

前端 `predict.js`: `runPrediction()`

- `POST /api/predict_all {num_points, time_step}`
- `views/api_predict.py: predict_all()`
- `services/predict_service.py: predict_all()`
 - 遍历 `detection_methods`
 - `predict_single()` → `clamp_params()` → `generate_prediction()`
 - `save_predict_data()` → `data/predict/pre*.dat`
- ← `{results: {methodId: {prediction, pred_times}}}`
- 前端渲染 `CatmullRom` 虚线 + `SphereGeometry` 球体

1.6.3 6.3 备份恢复流

用户点击"清理全部数据"

- `POST /api/clear_all_data`
- `data_service.clear_all_data()`
 - `backup_service.create_backup("auto")`
 - 复制 `data/fact/` → `snapshot/fact/`
 - 复制 `data/predict/` → `snapshot/predict/`
 - `dump detection_methods` → `snapshot/memory/<id>.dat`
 - 写 `manifest.json`
 - `os.remove` 删除 `data/fact/*`, `data/predict/*`
 - 清空 `detection_methods` 内存
- ← `{backup_name: "20260629_143052_auto"}`

用户恢复:

- `POST /api/restore_backup {backup_file}`
- `backup_service.restore_backup()`
 - `_clear_all()` # 先清空一切
 - 复制 `snapshot/fact/` → `data/fact/`
 - 复制 `snapshot/predict/` → `data/predict/`
 - 加载 `snapshot/memory/<id>.dat` → `detection_methods`

1.7 7. API 完整参考 (13 个端点)

所有响应: `{"success": true, ...}` 或 `{"success": false, "error": "..."}`

1.7.1 数据管理

端点	说明
<code>POST /api/refresh_data</code>	重载 <code>data/fact/</code> , 保留自选
<code>POST /api/load_data</code>	<code>multipart: file + method_id=self</code>

端点	说明
POST /api/clear_all_data	备份后清空, 返回 backup_name

1.7.2 预测 & AI

端点	请求体
POST /api/predict	{method_id, points, timestamps?, num_points?, time_step?}
POST /api/predict_all	{num_points?, time_step?}
POST /api/ai_suggestion	{methods_data: {id: {name, points, timestamps}}}

1.7.3 备份管理

端点	说明
GET /api/list_backups	返回 [{filename, timestamp, label, method, point_count}]
POST /api/restore_backup	{backup_file: " 快照名"}
POST /api/restore_all_backups	恢复最新
POST /api/backup/create	手动创建
POST /api/backup/delete	{backup_name: " 快照名"}

1.7.4 其他

端点	说明
POST /api/save_report	{content, platforms?} → reports/
GET /analysis/data	速度/加速度/曲率/高度序列

1.8 8. 添加新功能指南

1.8.1 8.1 新 API 端点

在 views/api.py 中添加 (或新建蓝图文件后注册):

```
@api_bp.route('/api/my_feature', methods=['POST'])
def my_feature():
    data = request.get_json(silent=True) or {}
    if not data.get('required_param'):
        return jsonify({'success': False, 'error': '缺少参数'}), 400
    result = some_service.do_work(data)
    return jsonify({'success': True, 'result': result})
```

1.8.2 8.2 新页面

1. frontend/pages/new.html: {% extends "base.html" %} + {% block content %} + {% block scripts %}
2. frontend/static/js/pages/new.js: ES6 模块
3. views/main.py: @main_bp.route('/new') → render_template('new.html', ...)
4. base.html 导航栏添加

1.8.3 8.3 新算法

在 core/ 下新建模块, 保持纯函数, 然后在 services/ 编排, views/ 暴露。

1.8.4 8.4 新检测平台

1. config.json 的 detection_methods 添加条目
2. data/fact/ 放入对应.dat 文件
3. core/io/data_loader.py 的 load_default_data() 添加文件映射

1.9 9. 前端开发参考

CSS 令牌 (main.css :root):

```
--blue: #0A84FF; --red: #FF453A; --green: #30D158;  
--bg-card: #2C2C2E; --bg-input: #3A3A3C;  
--radius-2xl: 24px; --radius-pill: 980px;  
--shadow-lg: 0 8px 24px rgba(0,0,0,0.4);  
--spring: cubic-bezier(0.175, 0.885, 0.32, 1.275);
```

Toast 通知:

```
import { toast } from '../common/toast.js';  
toast.success('操作成功');  
toast.error('操作失败');
```

共享函数:

```
import { lerp } from '../common/utils.js';  
const pos = lerp(points, timestamps, t); // 按时间线性插值坐标
```

备份列表: 每行复选框 + 信息, 底部 [恢复选中] [删除选中] [关闭], 右上角 [全选]。

1.10 10. 调试

查看所有路由

```
python -c "
```

```
from trajectory_reconstruction.app import create_app  
for r in sorted(create_app().url_map.iter_rules(), key=lambda x: x.rule):  
    print(f'{r.rule:40s} {sorted(r.methods)}')
```

查看全局状态

```
python -c "  
from trajectory_reconstruction.core.state import detection_methods  
for mid, d in detection_methods.items():  
    print(f'{mid}: {len(d["points\"]) } pts, visible={d["visible\"]}')"  
  
# 查看备份内容  
ls -R data/backup/ && cat data/backup/*/manifest.json  
  
# 重置环境  
rm -rf data/backup/* data/predict/* reports/* && git restore data/fact/
```

1.11 11. 代码规范

- **Python:** 类型注解, 导入顺序: 标准库 → 第三方 → 项目模块
- **JavaScript:** ES6 模块, async/await, 无全局变量
- **CSS:** 所有值使用 var(--xxx), 不硬编码
- **分层:** core 不导入 Flask/services/views; services 不导入 Flask/views
- **提交:** 中文描述, 简洁准确